

CHAPTER 8

Terraform Workspaces & Environments

Managing Dev, Staging & Prod from a Single Terraform Configuration

Instructor Deck • Terraform Fundamentals Series



What Are Workspaces?

An **isolated state environment** within the same Terraform project.

- Terraform ships with a default workspace out of the box
- Additional workspaces can be created and switched dynamically
- Each workspace keeps its own state file, preventing conflicts

● **default** *isolated .tfstate*

● **dev** *isolated .tfstate*

● **staging** *isolated .tfstate*

● **prod** *isolated .tfstate*

Why Use Workspaces — And When Not To



Why Use Workspaces

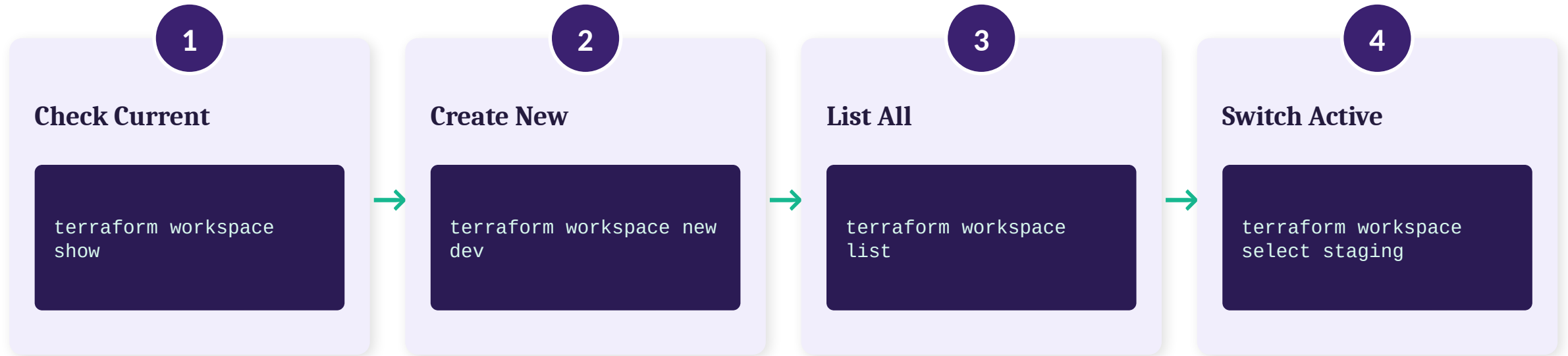
- ✓ Simplifies multi-environment deployments
- ✓ Reduces duplication — no separate Terraform projects needed
- ✓ Keeps state files isolated for each environment



When NOT to Use Workspaces

- ✗ Deploying to different AWS accounts — use separate backend configs instead
- ✗ Very large environments — separate Terraform projects may scale better

Managing Workspaces: The Core Commands



Sample output of terraform workspace list :

```
default
* dev
staging
prod
```

(* marks the active workspace)



Each command targets the active workspace's state — switching workspaces never touches another environment's resources.

8.2 STEP 5

The terraform.workspace Built-in Variable

Use **terraform.workspace** to dynamically adjust resources based on the active workspace.

```
resource "aws_instance" "web" {  
  ami          = "ami-0c55b159cbfafa1f0"  
  instance_type = "t2.micro"  
  
  tags = {  
    Name          = "web-${terraform.workspace}"  
    Environment = terraform.workspace  
  }  
}
```



Applied in the staging workspace:

Name = web-staging

Environment = staging

Instructor tip: run this same config in dev/prod and show learners how tags shift automatically.

Workspace-Specific Configuration with lookup()

```
locals {  
  instance_type_map = {  
    "default" = "t2.micro"  
    "dev"      = "t3.micro"  
    "staging" = "t3.medium"  
    "prod"    = "t3.large"  
  }  
}  
  
instance_type = lookup(  
  local.instance_type_map,  
  terraform.workspace, "t2.micro"  
)
```

Resolved instance_type per workspace

☐	default	t2.micro
☐	dev	t3.micro
☒	staging	t3.medium
☐	prod	t3.large

Managing Dev, Staging & Production



Use Workspaces for Small Environments

If your environments are simple, workspaces are a great fit.



Separate Backends for Separate AWS Accounts

Workspaces are not meant to manage separate cloud accounts.



Store State Remotely

Use an S3 backend (with DynamoDB locking) to avoid losing state.



Automate with CI/CD

Automate workspace selection in GitLab, Jenkins, or GitHub Actions.

SCENARIO

Goal: Deploy EC2 in Dev, Staging & Prod

Project structure

terraform-workspaces/

└─ main.tf	<i>Terraform config</i>
└─ variables.tf	<i>Variables file</i>
└─ backend.tf	<i>Remote state storage</i>
└─ outputs.tf	<i>Output file</i>

backend.tf — remote state in S3

```
terraform {  
  backend "s3" {  
    bucket = "my-terraform-state-bucket"  
    key    = "terraform-workspaces/  
            terraform.tfstate"  
    region = "us-east-1"  
    encrypt = true  
    dynamodb_table = "terraform-lock"  
  }  
}  
  
# Ensures each workspace has its own  
# isolated state file in S3
```


Switch Between Environments & Deploy

```
$ terraform init

$ terraform workspace select dev
$ terraform apply -auto-approve

$ terraform workspace select staging
$ terraform apply -auto-approve

$ terraform workspace select prod
$ terraform apply -auto-approve
```



web-dev

Environment = dev

t3.micro



web-staging

Environment = staging

t3.medium



web-prod

Environment = prod

t3.large

What We Covered



What workspaces are and why they matter



Creating, switching, and managing workspaces



Using `terraform.workspace` in configurations



Best practices for multi-environment deployments



Scenario: deploying EC2 across dev, staging & prod